

4. 大厂算法真题讲解（1/2）

1. 课程目标



- 初级：
 - 掌握双指针、滑动窗口和二叉树的基础概念；
- 中级：
 - 掌握双指针、滑动窗口和二叉树的常见面试题实现思路；
- 高级：
 - 掌握双指针、滑动窗口和二叉树面试题的JavaScript的完整代码实现；

2. 课程大纲

- 双指针
- 滑动窗口
- 二叉树

3. 双指针

3.1 最接近的三数之和

给定一个包括 n 个整数的数组 `nums` 和一个目标值 `target`。找出 `nums` 中的三个整数，使得它们的和与 `target` 最接近。返回这三个数的和。假定每组输入只存在唯一答案。

- 1 示例：
- 2
- 3 输入：`nums = [-1,2,1,-4]`，`target = 1`
- 4 输出：2
- 5 解释：与 `target` 最接近的和是 2 ($-1 + 2 + 1 = 2$)。

提示：

```
3 <= nums.length <= 10^3
```

```
-10^3 <= nums[i] <= 10^3
```

`-10^4 <= target <= 10^4`

链接: <https://leetcode-cn.com/problems/3sum-closest>

先按照升序排序, 然后分别从左往右依次选择一个基础点 `i` (`0 <= i <= nums.length - 3`), 在基础点的右侧用双指针去不断的找最小的差值。

假设基础点是 `i`, 初始化的时候, 双指针分别是:

- `left`: `i + 1`, 基础点右边一位。
- `right`: `nums.length - 1` 数组最后一位。

然后求此时的和, 如果和大于 `target`, 那么可以把右指针左移一位, 去试试更小一点的值, 反之则把左指针右移。

在这个过程中, 不断更新全局的最小差值 `min`, 和此时记录下来的和 `res`。

最后返回 `res` 即可。

```
1  /**
2   * @param {number[]} nums
3   * @param {number} target
4   * @return {number}
5   */
6  let threeSumClosest = function (nums, target) {
7      let n = nums.length
8      if (n === 3) {
9          return getSum(nums)
10     }
11     // 先升序排序 此为解题的前置条件
12     nums.sort((a, b) => a - b)
13
14     let min = Infinity // 和 target 的最小差
15     let res
16
17     // 从左往右依次尝试定一个基础指针 右边至少再保留两位 否则无法凑成3个
18     for (let i = 0; i <= nums.length - 3; i++) {
19         let basic = nums[i]
20         let left = i + 1; // 左指针先从 i 右侧的第一位开始尝试
21         let right = n - 1 // 右指针先从数组最后一项开始尝试
22
23         while (left < right) {
24             let sum = basic + nums[left] + nums[right] // 三数求和
25             // 更新最小差
26             let diff = Math.abs(sum - target)
27             if (diff < min) {
28                 min = diff
```

```

29         res = sum
30     }
31     if (sum < target) {
32         // 求出的和如果小于目标值的话 可以尝试把左指针右移 扩大值
33         left++
34     } else if (sum > target) {
35         // 反之则右指针左移
36         right--
37     } else {
38         // 相等的话 差就为0 一定是答案
39         return sum
40     }
41 }
42 }
43
44 return res
45 };
46
47 function getSum(nums) {
48     return nums.reduce((total, cur) => total + cur, 0)
49 }

```

3.2 通过删除字母匹配到字典里最长单词

给定一个字符串和一个字符串字典，找到字典里面最长的字符串，该字符串可以通过删除给定字符串的某些字符来得到。如果答案不止一个，返回长度最长且字典顺序最小的字符串。如果答案不存在，则返回空字符串。

```

1  示例 1:
2
3  输入:
4  s = "abpcplea", d = ["ale","apple","monkey","plea"]
5
6  输出:
7  "apple"
8  示例 2:
9
10 输入:
11 s = "abpcplea", d = ["a","b","c"]
12
13 输出:
14 "a"
15 说明:
16
17 所有输入的字符串只包含小写字母。

```

- 18 字典的大小不会超过 1000。
- 19 所有输入的字符串长度不会超过 1000。

链接: <https://leetcode-cn.com/problems/longest-word-in-dictionary-through-deleting>

为数组 `d` 中的**每个**单词分别生成**指针**，指向那个单词目前**已经匹配到**的字符下标。然后对 `s` 进行一次遍历，每轮循环中都对 `d` 中所有指针的**下一位**进行匹配，如果匹配到了，则那个单词的指针后移。一旦某个单词指针移动到那个字母的最后一位了，就和全局的 `find` 变量比较，先比较长度，后比较字典序，记录下来，最后返回即可。

```
1  /**
2   * @param {string} s
3   * @param {string[]} d
4   * @return {string}
5   */
6  let findLongestWord = function (s, d) {
7      let n = d.length
8      let points = Array(n).fill(-1)
9
10     let find = ""
11     for (let i = 0; i < s.length; i++) {
12         let char = s[i]
13         for (let j = 0; j < n; j++) {
14             let targetChar = d[j][points[j] + 1]
15             if (char === targetChar) {
16                 points[j]++
17                 let word = d[j]
18                 let wl = d[j].length
19                 if (points[j] === wl - 1) {
20                     let fl = find.length
21                     if (wl > fl || (wl === fl && word < find)) {
22                         find = word
23                     }
24                 }
25             }
26         }
27     }
28
29     return find
30 }
```

3.3 搜索二维矩阵

编写一个高效的算法来搜索 $m \times n$ 矩阵 matrix 中的一个目标值 target。该矩阵具有以下特性：

每行的元素从左到右升序排列。

每列的元素从上到下升序排列。

示例：

现有矩阵 matrix 如下：

```
1  [
2    [1, 4, 7, 11, 15],
3    [2, 5, 8, 12, 19],
4    [3, 6, 9, 16, 22],
5    [10, 13, 14, 17, 24],
6    [18, 21, 23, 26, 30],
7  ]
```

给定 target = 5，返回 true。

给定 target = 20，返回 false。

链接：<https://leetcode-cn.com/problems/search-a-2d-matrix-ii>

由题可得，当某一个单元格的值大于目标值的时候，那么目标值一定在这个单元格**上方**的行里。

如果单元格的值小于目标值的时候，那么目标值一定在这个单元格**右边**的列里。

根据这一特性，从最左下角那一格开始，设立**行指针** `row = y - 1`，设立**列指针** `column = 0`，然后不断根据上面描述特征移动这两个指针，直到找到目标值位置。如果超出边界还没有找到目标值，那么就返回 false。

```
1  /**
2   * @param {number[][]} matrix
3   * @param {number} target
4   * @return {boolean}
5   */
6  let searchMatrix = function (matrix, target) {
7      let y = matrix.length
8      if (!y) return false
9      let x = matrix[0].length
10
11     let row = y - 1
12     let column = 0
13     while (row >= 0 && column < x) {
14         let val = matrix[row][column]
15         if (val > target) {
16             row--
```

```
17     } else if (val < target) {
18         column++
19     } else if (val === target) {
20         return true
21     }
22 }
23 return false
24 }
```

3.4 判断子序列

给定字符串 s 和 t ，判断 s 是否为 t 的子序列。

你可以认为 s 和 t 中仅包含英文小写字母。字符串 t 可能会很长（长度 $\sim 500,000$ ），而 s 是个短字符串（长度 ≤ 100 ）。

字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，"ace"是"abcde"的一个子序列，而"aec"不是）。

```
1  示例 1:
2  s = "abc", t = "ahbgdc"
3
4  返回 true.
5
6  示例 2:
7  s = "axc", t = "ahbgdc"
8
9  返回 false.
```

后续挑战：

如果有大量输入的 S ，称作 $S_1, S_2, \dots, S_k$ 其中 $k \geq 10$ 亿，你需要依次检查它们是否为 T 的子序列。在这种情况下，你会怎样改变代码？

链接：<https://leetcode-cn.com/problems/is-subsequence>

判断字符串 s 是否是字符串 t 的子序列，我们可以建立一个 i 指针指向「当前 s 已经在 t 中成功匹配的字符下标后一位」。之后开始遍历 t 字符串，每当在 t 中发现 i 指针指向的目标字符时，就可以把 i 往后前进一位。

- 一旦 `i === s.length`，就代表 s 中的字符串全部按顺序在 t 中找到了，返回 `true`。
- 当遍历 t 结束后，就返回 `false`，因为 i 此时并没有成功走 t 的最后一位。

```

1  /**
2   * @param {string} s
3   * @param {string} t
4   * @return {boolean}
5   */
6  let isSubsequence = function (s, t) {
7      let sl = s.length
8      if (!sl) {
9          return true
10     }
11
12     let i = 0
13     for (let j = 0; j < t.length; j++) {
14         let target = s[i]
15         let cur = t[j]
16         if (cur === target) {
17             i++
18             if (i === sl) {
19                 return true
20             }
21         }
22     }
23
24     return false
25 };

```

3.5 分发饼干

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。对每个孩子 i ，都有一个胃口值 g_i ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 s_j 。如果 $s_j \geq g_i$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是尽可能满足越多数量的孩子，并输出这个最大数值。

注意：

你可以假设胃口值为正。

一个小朋友最多只能拥有一块饼干。

```

1  示例 1:
2
3  输入: [1,2,3], [1,1]
4
5  输出: 1
6
7  解释:

```

```
8  你有三个孩子和两块小饼干，3个孩子的胃口值分别是：1,2,3。
9  虽然你有两块小饼干，由于他们的尺寸都是1，你只能让胃口值是1的孩子满足。
10 所以你应该输出1。
11 示例 2：
12
13 输入：[1,2], [1,2,3]
14
15 输出：2
16
17 解释：
18 你有两个孩子和三块小饼干，2个孩子的胃口值分别是1,2。
19 你拥有的饼干数量和尺寸都足以让所有孩子满足。
20 所以你应该输出2。
```

链接：<https://leetcode-cn.com/problems/assign-cookies>

把饼干和孩子的需求都排序好，然后从最小的饼干分配给需求最小的孩子开始，不断的尝试新的饼干和新的孩子，这样能保证每个分给孩子的饼干都恰到好处的不浪费，又满足需求。

利用双指针不断的更新 `i` 孩子的需求下标和 `j` 饼干的值，直到两者有其一达到了终点位置：

1. 如果当前的饼干不满足孩子的胃口，那么把 `j++` 寻找下一个饼干，不用担心这个饼干被浪费，因为这个饼干更不可能满足下一个孩子（胃口更大）。
2. 如果满足，那么 `i++`；`j++`；`count++` 记录当前的成功数量，继续寻找下一个孩子和下一个饼干。

```
1  /**
2   * @param {number[]} g
3   * @param {number[]} s
4   * @return {number}
5   */
6  let findContentChildren = function (g, s) {
7      g.sort((a, b) => a - b)
8      s.sort((a, b) => a - b)
9
10     let i = 0
11     let j = 0
12
13     let count = 0
14     while (j < s.length && i < g.length) {
15         let need = g[i]
16         let cookie = s[j]
17
18         if (cookie >= need) {
```



```

19         count++
20         i++
21         j++
22     } else {
23         j++
24     }
25 }
26
27 return count
28 }

```

3.6 验证回文串

给定一个字符串，验证它是否是回文串，只考虑字母和数字字符，可以忽略字母的大小写。

说明：本题中，我们将空字符串定义为有效的回文串。

示例 1:

```

1  输入: "A man, a plan, a canal: Panama"
2  输出: true

```

示例 2:

```

1  输入: "race a car"
2  输出: false

```

<https://leetcode-cn.com/problems/valid-palindrome>

先根据题目给出的条件，通过正则把不匹配字符去掉，然后转小写。

建立双指针 `i`，`j` 分别指向头和尾，然后两个指针不断的向中间靠近，每前进一步就对比两端的字符串是否相等，如果不相等则直接返回 `false`。

如果直到 `i >= j` 也就是指针对撞了，都没有返回 `false`，那就说明符合「回文」的定义，返回 `true`。

```

1  /**
2   * @param {string} s
3   * @return {boolean}
4   */
5  let isPalindrome = function(s) {

```

```

6      s = s.replace(/^[^0-9a-zA-Z]/g, '').toLowerCase()
7      let i = 0
8      let j = s.length - 1
9
10     while(i < j) {
11         let head = s[i]
12         let tail = s[j]
13
14         if (head !== tail) {
15             return false
16         } else {
17             i++
18             j--
19         }
20     }
21     return true
22 };

```

3.7 合并两个有序数组

给你两个有序整数数组 `nums1` 和 `nums2`，请你将 `nums2` 合并到 `nums1` 中，使 `nums1` 成为一个有序数组。

说明:

初始化 `nums1` 和 `nums2` 的元素数量分别为 `m` 和 `n`。

你可以假设 `nums1` 有足够的空间（空间大小大于或等于 `m + n`）来保存 `nums2` 中的元素。

示例:

输入:

```

1  nums1 = [1,2,3,0,0,0], m = 3
2  nums2 = [2,5,6],         n = 3
3
4  输出: [1,2,2,3,5,6]

```

<https://leetcode-cn.com/problems/merge-sorted-array>

从后往前的双指针思路，先定义指针 `i` 和 `j` 分别指向数组中**有值的位置**的末尾，再定义指针 `k` 指向待填充的数组 1 的末尾。

然后不断的迭代 `i` 和 `j` 指针，如果 `i` 位置的值比 `j` 大，就移动 `i` 位置的值到 `k` 位置，反之亦然。

如果 i 指针循环完了，j 指针的数组里还有值未处理的话，直接从 k 位置开始向前填充 j 指针数组即可。因为此时数组 1 原本的值一定全部被填充到了数组 1 的后面位置，且这些值一定全部大于此时 j 指针数组里的值。

```
1  /**
2   * @param {number[]} nums1
3   * @param {number} m
4   * @param {number[]} nums2
5   * @param {number} n
6   * @return {void} Do not return anything, modify nums1 in-place instead.
7   */
8  let merge = function (arr1, m, arr2, n) {
9      // 两个指针指向数组非空位置的末尾
10     let i = m - 1;
11     let j = n - 1;
12     // 第三个指针指向第一个数组的末尾 填充数据
13     let k = arr1.length - 1;
14
15     while (i >= 0 && j >= 0) {
16         let num1 = arr1[i];
17         let num2 = arr2[j];
18
19         if (num1 > num2) {
20             arr1[k] = num1;
21             i--;
22         } else {
23             arr1[k] = num2;
24             j--;
25         }
26         k--;
27     }
28
29     while (j >= 0) {
30         arr1[k] = arr2[j];
31         j--;
32         k--;
33     }
34 };
```

3.8 移动零

给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。

示例:

```
1  输入: [0,1,0,3,12]
2  输出: [1,3,12,0,0]
```

说明:

必须在原数组上操作,不能拷贝额外的数组。尽量减少操作次数。

<https://leetcode-cn.com/problems/move-zeroes>

暴力法

先遍历一次,找出所有 0 的下标,然后删除掉所有 0 元素,再 push 相应的 0 的个数到末尾。

```
1  var moveZeroes = function (nums) {
2      let zeros = []
3      for (let i = 0; i < nums.length; i++) {
4          if (nums[i] === 0) {
5              zeros.push(i)
6          }
7      }
8      for (let j = zeros.length - 1; j >= 0; j--) {
9          nums.splice(zeros[j], 1)
10     }
11     for (let j = 0; j < zeros.length; j++) {
12         nums.push(0)
13     }
14     return nums
15 }
```

双指针

慢指针 j 从 0 开始,当快指针 i 遍历到非 0 元素的时候, i 和 j 位置的元素交换,然后把 j + 1。

也就是说,快指针 i 遍历完毕后, [0, j) 区间就存放着所有非 0 元素,而剩余的[j, n]区间再遍历一次,用 0 填满即可。

```
1  var moveZeroes = function (nums) {
2      let j = 0
3
4      for (let i = 0; i < nums.length; i++) {
5          if (nums[i] !== 0) {
6              nums[j] = nums[i]
```

```

7         j++
8     }
9 }
10
11 while (j < nums.length) {
12     nums[j] = 0
13     j++
14 }
15 }

```

双指针（交换位置）

在上面的算法里，快指针遍历完成后，还要遍历慢指针到末尾来填充 0。实际上这题只要遇到非 0 元素，就把当前位置的值和慢指针位置 j 的值交换，然后只有此时 j 才 + 1，即可完成。

```

1  var moveZeroes = function (nums) {
2      let j = 0
3
4      for (let i = 0; i < nums.length; i++) {
5          if (nums[i] !== 0) {
6              swap(nums, i, j)
7              j++
8          }
9      }
10 }
11
12 function swap(nums, i, j) {
13     let temp = nums[i]
14     nums[i] = nums[j]
15     nums[j] = temp
16 }

```

4. 滑动窗口

4.1 滑动窗口的最大值

滑动窗口的最大值

科技馆内有一台虚拟观景望远镜，它可以用来观测特定纬度地区的地形情况。该纬度的海拔数据记于数组 **nums**，其中 **nums** `[i]` 表示对应位置的海拔高度。请找出并返回望远镜视野范围 k 内，可以观测到的最高海拔值。

示例:

```

1  输入: nums = [14,2,27,-5,28,13,39], k = 3
2  输出: [27,27,28,28,39]
3  解释:
4      滑动窗口的位置                最大值
5  -----
6  [14 2 27] -5 28 13 39                27
7  14 [2 27 -5] 28 13 39                27
8  14 2 [27 -5 28] 13 39                28
9  14 2 27 [-5 28 13] 39                28
10 14 2 27 -5 [28 13 39]                39

```

提示:

你可以假设 k 总是有效的, 在输入数组不为空的情况下, $1 \leq k \leq$ 输入数组的大小。

链接: <https://leetcode.cn/problems/hua-dong-chuang-kou-de-zui-da-zhi-lcof/description/>

滑动窗口, 每次左右各滑动一位, 并且求窗口中的最大值记录即可, 这题坑的地方在于边界情况比较多。

```

1  let maxSlidingWindow = function (nums, k) {
2      if (k === 0 || !nums.length) {
3          return []
4      }
5      let left = 0
6      let right = k - 1
7      let res = [findMax(nums, left, right)]
8
9      while (right < nums.length - 1) {
10         right++
11         left++
12         res.push(findMax(nums, left, right))
13     }
14
15     return res
16 }
17
18 function findMax(nums, left, right) {
19     let max = -Infinity
20     for (let i = left; i <= right; i++) {
21         max = Math.max(max, nums[i])
22     }
23     return max
24 }

```

4.2 找到字符串中所有字母异位词

给定一个字符串 s 和一个非空字符串 p ，找到 s 中所有是 p 的字母异位词的子串，返回这些子串的起始索引。

字符串只包含小写英文字母，并且字符串 s 和 p 的长度都不超过 20100。

说明：

字母异位词指字母相同，但排列不同的字符串。

不考虑答案输出的顺序。

示例 1:

```
1  输入：
2  s: "cbaebabacd" p: "abc"
3
4  输出：
5  [0, 6]
6
7  解释：
8  起始索引等于 0 的子串是 "cba"，它是 "abc" 的字母异位词。
9  起始索引等于 6 的子串是 "bac"，它是 "abc" 的字母异位词。
```

示例 2:

```
1  输入：
2  s: "abab" p: "ab"
3
4  输出：
5  [0, 1, 2]
6
7  解释：
8  起始索引等于 0 的子串是 "ab"，它是 "ab" 的字母异位词。
9  起始索引等于 1 的子串是 "ba"，它是 "ab" 的字母异位词。
10 起始索引等于 2 的子串是 "ab"，它是 "ab" 的字母异位词。
```

链接：<https://leetcode-cn.com/problems/find-all-anagrams-in-a-string>

还是典型的滑动窗口去解决的问题，由于字母异位词一定是长度相等的，所以我们需要把窗口的长度始终维持在目标字符的长度，也就是说，每次循环结束后 `left` 和 `right` 是同步前进的。

由于字母异位词不需要考虑顺序，所以只需要运用一个辅助函数 `isAnagrams` 来判断两个 `map` 中记录的字母次数，即可判断出当前位置开始的子串是否和目标字符串形成字母异位词。

```
1  /**
2   * @param {string} s
3   * @param {string} p
4   * @return {number[]}
5   */
6  let findAnagrams = function (s, p) {
7      let targetMap = makeCountMap(p)
8      let sl = s.length
9      let pl = p.length
10     // [left,...right] 滑动窗口
11     let left = 0
12     let right = pl - 1
13     let windowMap = makeCountMap(s.substring(left, right + 1))
14     let res = []
15
16     while (left <= sl - pl && right < sl) {
17         if (isAnagrams(windowMap, targetMap)) {
18             res.push(left)
19         }
20         windowMap[s[left]]--
21         right++
22         left++
23         addCountToMap(windowMap, s[right])
24     }
25
26     return res
27 }
28
29 let isAnagrams = function (windowMap, targetMap) {
30     let targetKeys = Object.keys(targetMap)
31     for (let targetKey of targetKeys) {
32         if (
33             !windowMap[targetKey] ||
34             windowMap[targetKey] !== targetMap[targetKey]
35         ) {
36             return false
37         }
38     }
39     return true
40 }
41
42 function addCountToMap(map, str) {
43     if (!map[str]) {
```



```

44     map[str] = 1
45   } else {
46     map[str]++
47   }
48 }
49
50 function makeCountMap(strs) {
51   let map = {}
52   for (let i = 0; i < strs.length; i++) {
53     let letter = strs[i]
54     addCountToMap(map, letter)
55   }
56   return map
57 }

```

4.3 最小覆盖子串

给你一个字符串 S、一个字符串 T，请在字符串 S 里面找出：包含 T 所有字符的最小子串。

示例：

```

1  输入：S = "ADOBECODEBANC", T = "ABC"
2  输出："BANC"

```

说明：

如果 S 中不存在这样的子串，则返回空字符串 ""。

如果 S 中存在这样的子串，我们保证它是唯一的答案。

<https://leetcode-cn.com/problems/minimum-window-substring>

根据目标字符串 `t` 生成一个目标 map，记录每个字符的目标值出现的次数。

然后就是维护一个滑动窗口，并且针对这个滑动窗口中的字符也生成一个 map 去记录字符出现次数。

每次循环都去对比窗口的 map 里的字符是否能覆盖目标 map 里的字符。

覆盖的意思就是，目标 map 里的每个字符在窗口 map 中出现，并且出现的次数要 $\geq$ 目标 map 中此字符出现的次数。

窗口滑动逻辑：

1. 如果当前还没有能覆盖，那么就右滑右边界。
2. 如果当前已经覆盖了，记录下当前的子串，并且右滑左边界看看能否进一步缩小子串的长度。

两种情况下停止循环，返回结果：

1. 左边界达到 给定字符长度 - 目标字符的长度，此时不管匹配与否，都是最短能满足的了。
2. 右边界超出给定字符的长度，这种情况会出现在右边界已经到头了，但是还没有能覆盖目标字符串，此时就算继续滑动也不可能得到结果了。

```
1  /**
2   * @param {string} s
3   * @param {string} t
4   * @return {string}
5   */
6  let minWindow = function (s, t) {
7      // 先制定目标 根据t字符串统计出每个字符应该出现的个数
8      let targetMap = makeCountMap(t)
9
10     let sl = s.length
11     let tl = t.length
12     let left = 0 // 左边界
13     let right = -1 // 右边界
14     let countMap = {} // 当前窗口子串中 每个字符出现的次数
15     let min = "" // 当前计算出的最小子串
16
17     // 循环终止条件是两者有一者超出边界
18     while (left <= sl - tl && right <= sl) {
19         // 和 targetMap 对比出现次数 确定是否满足条件
20         let isValid = true
21         Object.keys(targetMap).forEach((key) => {
22             let targetCount = targetMap[key]
23             let count = countMap[key]
24             if (!count || count < targetCount) {
25                 isValid = false
26             }
27         })
28
29         if (isValid) {
30             // 如果满足 记录当前的子串 并且左边界右移
31             let currentValidLength = right - left + 1
32             if (currentValidLength < min.length || min === "") {
33                 min = s.substring(left, right + 1)
34             }
35             // 也要把map里对应的项去掉
36             countMap[s[left]]--
37             left++
38         } else {
39             // 否则右边界右移
40             addCountToMap(countMap, s[right + 1])
41             right++

```

```

42     }
43 }
44
45     return min
46 }
47
48 function addCountToMap(map, str) {
49     if (!map[str]) {
50         map[str] = 1
51     } else {
52         map[str]++
53     }
54 }
55
56 function makeCountMap(strs) {
57     let map = {}
58     for (let i = 0; i < strs.length; i++) {
59         let letter = strs[i]
60         addCountToMap(map, letter)
61     }
62     return map
63 }
64
65 console.log(minWindow("aa", "a"))

```

4.4 无重复字符的最长子串

给定一个字符串，请你找出其中不含有重复字符的 **最长子串** 的长度。

示例 1:

- 1 输入: "abcabcbb"
- 2 输出: 3
- 3 解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。

示例 2:

- 1 输入: "bbbbbb"
- 2 输出: 1
- 3 解释: 因为无重复字符的最长子串是 "b", 所以其长度为 1。

示例 3:

```
1  输入: "pwwkew"
2  输出: 3
3  解释: 因为无重复字符的最长子串是 "wke", 所以其长度为 3。
4      请注意, 你的答案必须是 子串 的长度, "pwke" 是一个子序列, 不是子串。
```

链接: <https://leetcode-cn.com/problems/longest-substring-without-repeating-characters>

这题是比较典型的滑动窗口问题, 定义一个左边界 `left` 和一个右边界 `right`, 形成一个窗口, 并且在这个窗口中保证不出现重复的字符串。

这需要用到一个新的变量 `freqMap`, 用来记录窗口中的字母出现的频率数。在此基础上, 先尝试取窗口的右边界再右边一个位置的值, 也就是 `str[right + 1]`, 然后拿这个值去 `freqMap` 中查找:

1. 这个值没有出现过, 那就直接把 `right ++`, 扩大窗口右边界。
2. 如果这个值出现过, 那么把 `left ++`, 缩进左边界, 并且记得把 `str[left]` 位置的值在 `freqMap` 中减掉。

循环条件是 `left < str.length`, 允许左边界一直滑动到字符串的右界。

```
1  /**
2   * @param {string} s
3   * @return {number}
4   */
5  let lengthOfLongestSubstring = function (str) {
6      let n = str.length
7      // 滑动窗口为s[left...right]
8      let left = 0
9      let right = -1
10     let freqMap = {} // 记录当前子串中下标对应的出现频率
11     let max = 0 // 找到的满足条件子串的最长长度
12
13     while (left < n) {
14         let nextLetter = str[right + 1]
15         if (!freqMap[nextLetter] && nextLetter !== undefined) {
16             freqMap[nextLetter] = 1
17             right++
18         } else {
19             freqMap[str[left]] = 0
20             left++
21         }
22         max = Math.max(max, right - left + 1)
23     }
```

```
24
25     return max
26 }
```

4.5 长度最小的子数组

给定一个含有 n 个正整数的数组和一个正整数 s ，找出该数组中满足其和 $\geq s$ 的长度最小的连续子数组，并返回其长度。如果不存在符合条件的连续子数组，返回 0。

示例:

```
1  输入: s = 7, nums = [2,3,1,2,4,3]
2  输出: 2
3  解释: 子数组 [4,3] 是该条件下的长度最小的连续子数组。
```

进阶:

如果你已经完成了 $O(n)$ 时间复杂度的解法, 请尝试 $O(n \log n)$ 时间复杂度的解法。

<https://leetcode-cn.com/problems/minimum-size-subarray-sum/submissions>

暴力法（优化）

纯暴力的循环也就是穷举每种子数组并求和，当然是会超时的，这里就不做讲解了。下面这种解法会在暴力法的基础上稍作优化，具体的思路如下：

1. 先选定下标 i 从 0 作为切分数组的起点，然后下标 j 作为数组的右边界从 0 开始不停向后扩展，每往后一位，就把本次的求和加上新的数字，只要本轮循环的和大于 s ，就应该停止循环，因为没必要再往后扩展了，往后扩展的数组长度一定是大于当前长度的。
2. 选定下标 1 为切分数组的起点，进入下一轮循环。

```
1  /**
2   * @param {number} s
3   * @param {number[]} nums
4   * @return {number}
5   */
6  let minSubArrayLen = function (s, nums) {
7      let min = Infinity
8      for (let i = 0; i < nums.length; i++) {
9          let sum = 0
10         for (let j = i; j < nums.length; j++) {
11             sum += nums[j]
12             if (sum >= s) {
```

```

13         min = Math.min(min, j - i + 1)
14         if (min === 1) {
15             return min
16         }
17         break
18     }
19 }
20 }
21 return min === Infinity ? 0 : min
22 }

```

滑动窗口

定义两个下标 i 、 j 为左右边界，中间的子数组为滑动窗口。在更新窗口的过程中不断的更新窗口之间的值的和 sum 。

1. 当 $sum < \text{目标值}$ ，说明值不够大， $j++$ ，右边界右移。
2. 当 $sum \geq \text{目标值}$ ，满足条件，把当前窗口的大小和记录的最小值进行对比，更新最小值。并且 $i++$ 左窗口右移，继续找最优解。

当 i 超出了数组的右边界，循环终止。

```

1  /**
2   * @param {number} s
3   * @param {number[]} nums
4   * @return {number}
5   */
6  let minSubArrayLen = function (s, nums) {
7      let n = nums.length
8      // 定义[i,...j]滑动窗口 取这个窗口里的和
9      let i = 0
10     let j = -1
11
12     let sum = 0
13     let res = Infinity
14
15     while (i < n) {
16         if (sum < s) {
17             sum += nums[++j]
18         } else {
19             sum -= nums[i]
20             i++
21         }
22
23         if (sum >= s) {
24             res = Math.min(res, j - i + 1)

```

```
25     }
26     }
27     return res === Infinity ? 0 : res
28 }
```

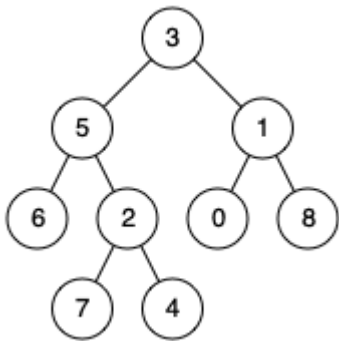
5. 二叉树

5.1 二叉树的最近公共祖先

给定一个二叉树，找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个结点 p、q，最近公共祖先表示为一个结点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

例如，给定如下二叉树：root = [3,5,1,6,2,0,8,null,null,7,4]



- 1 示例 1:
- 2 输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
- 3 输出: 3
- 4 解释: 节点 5 和节点 1 的最近公共祖先是节点 3。
- 5 示例 2:
- 6 输入: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
- 7 输出: 5
- 8 解释: 节点 5 和节点 4 的最近公共祖先是节点 5。因为根据定义最近公共祖先节点可以为节点本身。
- 9 说明:
- 10 所有节点的值都是唯一的。
- 11 p、q 为不同节点且均存在于给定的二叉树中。

链接: <https://leetcode-cn.com/problems/lowest-common-ancestor-of-a-binary-tree>

先通过 DFS 去找到 p 和 q 节点的深度，并且在查找的过程中对节点和他们的子节点之间建立父子关系。

之后，从深度最浅的那个节点开始（深度浅，离公共祖先一定更近）不断往上查找公共祖先即可。

```
1  let lowestCommonAncestor = function (root, p, q) {
2    let findAndCreate = (node, target, depth) => {
3      if (node !== target) {
4        let findInLeft
5        if (node.left) {
6          node.left.parent = node
7          findInLeft = findAndCreate(node.left, target, depth + 1)
8        }
9
10       if (findInLeft) {
11         return findInLeft
12       } else {
13         if (node.right) {
14           node.right.parent = node
15           return findAndCreate(node.right, target, depth + 1)
16         }
17       }
18     } else {
19       return {
20         depth,
21         node,
22       }
23     }
24   }
25
26   let findP = findAndCreate(root, p, 0) || {}
27   let findQ = findAndCreate(root, q, 0) || {}
28
29   let cur = findP.depth > findQ.depth ? findQ.node : findP.node
30
31   while (!(isAncestor(cur, p) && isAncestor(cur, q))) {
32     cur = cur.parent
33   }
34
35   return cur
36 }
37
38 function isAncestor(node, target) {
39   if (!node) {
40     return false
41   }
42   if (node !== target) {
43     return !(isAncestor(node.left, target) || isAncestor(node.right, target))
44   } else {
```



```
45     return true
46 }
47 }
```

5.2 将有序数组转换为二叉搜索树

将一个按照升序排列的有序数组，转换为一棵高度平衡二叉搜索树。

本题中，一个高度平衡二叉树是指一个二叉树每个节点的左右两个子树的高度差的绝对值不超过 1。

示例:

```
1  给定有序数组: [-10,-3,0,5,9],
2
3  一个可能的答案是: [0,-3,9,-10,null,5]，它可以表示下面这个高度平衡二叉搜索树:
4
5      0
6     / \
7    -3  9
8     /  /
9    -10 5
```

链接: <https://leetcode-cn.com/problems/convert-sorted-array-to-binary-search-tree>

将有序数组分为左、中、右三个部分，以中为根节点，并且递归的对左右两部分建立平衡二叉树即可。

<https://leetcode-cn.com/problems/convert-sorted-array-to-binary-search-tree/solution/tu-jie-er-cha-sou-suo-shu-gou-zao-di-gui-python-go>

```
1  let sortedArrayToBST = function (nums) {
2      let n = nums.length
3      if (!n) {
4          return null
5      }
6      let mid = Math.floor(n / 2)
7      let root = new TreeNode(nums[mid])
8
9      root.left = sortedArrayToBST(nums.slice(0, mid))
10     root.right = sortedArrayToBST(nums.slice(mid + 1, n))
11
12     return root
13 };
```

5.3 删除二叉搜索树中的节点

给定一个二叉搜索树的根节点 `root` 和一个值 `key`，删除二叉搜索树中的 `key` 对应的节点，并保证二叉搜索树的性质不变。返回二叉搜索树（有可能被更新）的根节点的引用。

一般来说，删除节点可分为两个步骤：

首先找到需要删除的节点；

如果找到了，删除它。

说明：要求算法时间复杂度为 $O(h)$ ， h 为树的高度。

示例：

```
1 root = [5,3,6,2,4,null,7]
```

```
2 key = 3
```

```
3
```

```
4      5
```

```
5     / \
```

```
6    3   6
```

```
7   / \   \
```

```
8  2  4   7
```

```
9
```

```
10 给定需要删除的节点值是 3，所以我们首先找到 3 这个节点，然后删除它。
```

```
11
```

```
12 一个正确的答案是 [5,4,6,2,null,null,7]，如下图所示。
```

```
13
```

```
14      5
```

```
15     / \
```

```
16    4   6
```

```
17   /     \
```

```
18  2       7
```

```
19
```

```
20 另一个正确答案是 [5,2,6,null,4,null,7]。
```

```
21
```

```
22      5
```

```
23     / \
```

```
24    2   6
```

```
25   \   \
```

```
26    4   7
```

```
27
```

链接：<https://leetcode-cn.com/problems/delete-node-in-a-bst>

先通过递归去找到目标节点的 **父节点** `parent`，和目标节点的**位置** `pos`：`left` 或 `right`，方便后续删除操作。

1. 待删除节点没有 `left` 或者 `right` 子节点的情况下，直接吧 `parent[pos] = null` 清空即可。
2. 待删除节点只有 `left` 或只有 `right`，那就把 `parent[pos]` 赋值为存在的那个节点即可。
3. 如果 `left` 和 `right` 都在的情况下，比较复杂一些，把待删除节点的左右子节点分别称为 `targetLeft` 和 `targetRight`，先找到 `targetRight` 的最左子节点，这个子节点一定在右子树中最小，但是在左子树中最大。然后把 `parent[pos]` 赋值为 `targetRight`，再把 `targetRight` 的最左下角子节点的 `left` 赋值成原来的 `targetLeft` 即可。

```
1  let deleteNode = function (root, key) {
2    let findNodePos = (node, key) => {
3      if (!node) {
4        return false
5      }
6      if (node.left && node.left.val === key) {
7        return {
8          parent: node,
9          pos: "left",
10         }
11       } else if (node.right && node.right.val === key) {
12         return {
13           parent: node,
14           pos: "right",
15         }
16       } else {
17         return findNodePos(node.left, key) || findNodePos(node.right, key)
18       }
19     }
20
21     let findLastLeft = (node) => {
22       if (!node.left) {
23         return node
24       }
25       return findLastLeft(node.left)
26     }
27
28     let virtual = new TreeNode()
29     virtual.left = root
30
31     let finded = findNodePos(virtual, key)
32     if (finded) {
33       let { parent, pos } = finded
```

```

34     let target = parent[pos]
35     let targetLeft = target.left
36     let targetRight = target.right
37
38     if (!targetLeft && !targetRight) {
39         parent[pos] = null
40     } else if (!targetRight) {
41         parent[pos] = targetLeft
42     } else if (!targetLeft) {
43         parent[pos] = targetRight
44     } else {
45         parent[pos] = targetRight
46         let lastLeft = findLastLeft(targetRight)
47         lastLeft.left = targetLeft
48     }
49 }
50
51 return virtual.left
52 }

```

5.4 路径总和

给定一个二叉树，它的每个结点都存放着一个整数值。

找出路径和等于给定数值的路径总数。

路径不需要从根节点开始，也不需要在叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

二叉树不超过 1000 个节点，且节点数值范围是 $[-1000000, 1000000]$ 的整数。

示例：

```

1  root = [10,5,-3,3,2,null,11,3,-2,null,1], sum = 8
2
3      10
4     /  \
5    5   -3
6   / \   \
7  3  2   11
8 / \   \
9 3 -2  1
10
11 返回 3。和等于 8 的路径有：
12
13 1. 5 -> 3
14 2. 5 -> 2 -> 1

```

链接: <https://leetcode-cn.com/problems/path-sum-iii>

实际上这题需要统计三个值:

1. 以 node 为起点, 包含这个 node 本身加起来等于 sum 的路径数量之和。
2. 以 node.left 为起点, 等于 sum 的路径数量之和。
3. 以 node.right 为起点, 等于 sum 的路径数量之和。

```

1  let pathSum = function (root, sum) {
2    if (!root) return 0
3    return (
4      countSum(root, sum) + pathSum(root.left, sum) + pathSum(root.right, sum)
5    )
6  }
7
8  let countSum = (node, sum) => {
9    let count = 0
10   let dfs = (node, target) => {
11     if (!node) return
12
13     // 这里拿到结果不能直接return 要继续向下考虑
14     // 比如下面的两个节点是 1, -1 那么它们相加得 0 也能得到一个路径
15     if (node.val === target) {
16       count += 1
17     }
18
19     dfs(node.left, target - node.val)
20     dfs(node.right, target - node.val)
21   }
22   dfs(node, sum)
23   return count
24 }
```

5.5 求根到叶子节点数字之和

给定一个二叉树, 它的每个结点都存放一个 0-9 的数字, 每条从根到叶子节点的路径都代表一个数字。

例如, 从根到叶子节点路径 1->2->3 代表数字 123。

计算从根到叶子节点生成的所有数字之和。

说明: 叶子节点是指没有子节点的节点。

```
1  示例 1:
2
3  输入: [1,2,3]
4      1
5     / \
6    2   3
7  输出: 25
8  解释:
9  从根到叶子节点路径 1->2 代表数字 12.
10 从根到叶子节点路径 1->3 代表数字 13.
11 因此, 数字总和 = 12 + 13 = 25.
12
13 示例 2:
14
15 输入: [4,9,0,5,1]
16      4
17     / \
18    9   0
19   / \
20  5   1
21 输出: 1026
22 解释:
23 从根到叶子节点路径 4->9->5 代表数字 495.
24 从根到叶子节点路径 4->9->1 代表数字 491.
25 从根到叶子节点路径 4->0 代表数字 40.
26 因此, 数字总和 = 495 + 491 + 40 = 1026.
```

链接: <https://leetcode-cn.com/problems/sum-root-to-leaf-numbers>

先按照字符串拼接的思路用 DFS 收集所有到达叶子节点的路径的集合, 最后把这些字符串转化为数字求和即可。

```
1  /**
2   * Definition for a binary tree node.
3   * function TreeNode(val) {
4   *     this.val = val;
5   *     this.left = this.right = null;
6   * }
7   */
8  /**
9   * @param {TreeNode} root
```

```

10     * @return {number}
11     */
12     let sumNumbers = function (root) {
13         let paths = []
14
15         let dfs = (node, path) => {
16             if (!node) {
17                 return
18             }
19
20             let newPath = `${path}${node.val}`
21             if (!node.left && !node.right) {
22                 paths.push(newPath)
23                 return
24             }
25
26             dfs(node.left, newPath)
27             dfs(node.right, newPath)
28         }
29
30         dfs(root, '')
31         return paths.reduce((total, val) => {
32             return total + Number(val)
33         }, 0)
34     };

```

5.6 平衡二叉树

给定一个二叉树，判断它是否是高度平衡的二叉树。

本题中，一棵高度平衡二叉树定义为：

一个二叉树每个节点的左右两个子树的高度差的绝对值不超过1。

示例 1:

```

1  给定二叉树 [3,9,20,null,null,15,7]
2
3      3
4     / \
5    9  20
6   /  \
7  15   7
8  返回 true 。
9
10 示例 2:
11

```

```
12 给定二叉树 [1,2,2,3,3,null,null,4,4]
13
14      1
15     /\
16    2  2
17   /\  /\
18  3  3 4  4
19 /\
20 4  4
21 返回 false 。
```

链接: <https://leetcode-cn.com/problems/balanced-binary-tree>

首先定义一个辅助函数 `getHeight` 用于获取二叉树某个节点的高度, 只需要递归即可:

```
1  function getHeight(node) {
2      if (!node) return 0
3      return Math.max(getHeight(node.left), getHeight(node.right)) + 1
4  }
```

之后是否平衡就比较好求了, 首先比较直接子节点 `left` 和 `right` 是否高度相差绝对值 ≤ 1 , 之后再递归的比较 `left` 和 `right` 是否是平衡二叉树即可。

```
1  var isBalanced = function (root) {
2      if (!root) {
3          return true
4      }
5
6      let isSonBalnaced = Math.abs(getHeight(root.left) -
getHeight(root.right)) <= 1
7
8      return isSonBalnaced && isBalanced(root.left) && isBalanced(root.right)
9  };
```